

Department of Computer and Software Engineering**SE: Machine Learning****Course Instructor:****Date:****Day:****Semester:****Lab 11: Refactoring, MLflow and Deployment**

Lab Title	CLO	BT	PLO	Weightage
Refactoring & Deployment MLflow, APIs, Serialization				

Name	Roll Number	Report /10	Viva /5	Total /15

Checked on: _____

Signature: _____

Objectives

- Refactor unstructured ML code into modular functions
- Track experiments using MLflow
- Serialize trained models
- Build REST APIs using FastAPI

Problem Statement

In real-world machine learning systems, models are not built as isolated scripts but as part of structured, maintainable, and deployable pipelines.

You are given a poorly written (“spaghetti”) machine learning script that mixes data loading, preprocessing, model training, and evaluation in a single block of code. Such code is difficult to maintain, extend, debug, and deploy.

Your task is to transform this code into a clean and modular pipeline by separating concerns into well-defined functions.

After refactoring, you will extend the pipeline to include:

- **Experiment Tracking:** Use MLflow to log model parameters, performance metrics, and artifacts.
- **Model Serialization:** Save the trained model to disk and reload it for inference.
- **Model Deployment:** Build a REST API using FastAPI to serve predictions from the trained model.

You will use the dataset from the previous lab (Lab 10) to ensure continuity.

By the end of this lab, you will have a complete mini machine learning system that includes:

- Clean, modular code
- Experiment tracking
- Persistent models
- A working prediction API

Part A: Refactoring

Task 1: Identify Issues

Open `train_bad.py` and list at least four problems related to structure and maintainability:

1. **Mixed Concerns:** I/O, preprocessing, and training are in the global scope, making deployment impossible without running the full script.
2. **Magic Numbers:** Hardcoded parameters like `max_depth=100` and `random_state=42` are baked into the execution.
3. **Data Leakage:** The target label was not properly isolated from the training features, leading to an artificially perfect accuracy.
4. **Dead Code & No Tracking:** Unused variables exist, categorical columns (`city`) are poorly handled, and no experiment metrics are preserved.

Task 2: Modularization

Refactored the script into isolated functions:

- `load_data()` — successfully handles purely I/O operations.
- `preprocess_data(df)` — strictly drops the categorical `city` column and isolates `X` from `y` to prevent data leakage.
- `train_model(X, y)` — fits the clean matrix without needing redundant data-type checks.
- `evaluate_model(model, X, y)` — utilizes `sklearn.metrics` to return the float accuracy.
- `run_pipeline()` — acts as the orchestrator to run the above pipeline in sequence.

Part B: MLflow

Task 3: Logging

Inside `run_pipeline()`, the training phase is wrapped in an MLflow context manager:

- **Parameters:** Logged `model_type` and `random_state` using `mlflow.log_param`.
- **Metrics:** Logged the evaluation float via `mlflow.log_metric("accuracy", acc)`.
- **Model:** Saved the model weights to the tracking server using `mlflow.sklearn.log_model`.

Part C: Serialization

Task 4: Save and Load Model

Bridged the gap between the training script and the web API using `joblib`:

- **Export:** Added `joblib.dump(model, "model.joblib")` at the end of the pipeline.
- **Import:** Added `joblib.load("model.joblib")` in the global scope of the FastAPI app, wrapped in a `try/except FileNotFoundError` block to prevent server crashes if booted before training.

Part D: FastAPI

Task 5: API Endpoint

Secured the `/predict` POST endpoint:

- **Defense 1:** Raises a 503 error if `model` is `None`.
- **Defense 2:** Raises a 422 error if the `"features"` key is missing from the JSON.
- **Matrix Conversion:** Applied `np.array().reshape(1, -1)` to translate the 1D JSON array into a 2D matrix required by Scikit-Learn for inference.

Discussion Questions

- **Why is modular design important?** It prevents the "monolith trap", allowing individual components (like I/O or training) to be tested, scaled, and debugged independently without breaking the entire system.
- **What are benefits of MLflow?** It acts as a version-control ledger for hyperparameter tuning, ensuring performance metrics and model artifacts are saved rather than lost when the RAM clears after a script finishes.
- **Why is model serialization required?** It freezes the trained model structure into a binary file, allowing a web server (like FastAPI) to load the "brain" instantly without needing to retrain the model on every startup.